

ACTIVATION SPARSIFICATION IN TRANSFORMERS: THRESHOLD PLACEMENT, PRESSURE PLACEMENT, AND SPARSE EXECUTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation sparsity can accelerate language-model inference when specialized kernels skip computation associated with zero activations. Existing methods obtain sparsity through different combinations of training-time pressure, thresholding, and placement, but their relative and combined effects have not been isolated under a common setup. As a result, it remains unclear which interventions drive the gains, when they are complementary or redundant, and how much model-wide computation they actually expose to sparsification. We study these interventions under matched pretraining conditions across Pythia models. To measure their computational potential, we define model-wide activation sparsity $\mathcal{S}_{\text{model}}$ as the fraction of forward matrix-multiplication products with a zero activation operand, together with an architecture-dependent sparsity ceiling $\mathcal{S}_{\text{model}}^{\text{max}}$. Pressure placement changes the quality–sparsity trade-off: in the 14M sweep, h-only pressure gives favorable moderate-threshold operating points, while broad seven-site pressure becomes useful for extreme sparsity. The broad-pressure recipes reach 27.48%, 40.60%, and 80.62% $\mathcal{S}_{\text{model}}$ at 14M, 70M, and 410M parameters, corresponding to 0.918, 0.822, and 0.924 of their respective sparsity ceilings. Selected trained recipes also outperform uniform post-hoc thresholding at high sparsity across all model sizes. For 35 measured 14M checkpoints on an RTX5090, structured projection work bypass is more strongly associated with projection-skipping gain ($R^2 = 0.941$) than scalar projection sparsity ($R^2 = 0.425$); larger logical sparsity need not imply lower optimized latency. These results show that activation sparsification is governed by the interaction between how zeros are induced, where they are placed, and how much of that zero structure can be exploited by sparse execution.

1 INTRODUCTION

Activation sparsity can improve language-model inference efficiency when specialized kernels skip computations associated with zero activations. Recent work has shown that sparsity can be exposed in pretrained models through post-hoc thresholding or induced during training, in both cases with measured inference speedups (Liu et al., 2024; Cetin et al., 2026). These results raise a practical design question: how to effectively sparsify a dense model in activations into an exploitable structure for optimized inference?

Existing approaches induce activation sparsity in different ways. TEAL applies post-hoc clipping to pretrained models, ProSparse uses ℓ_1 regularization on feed-forward (FFN) activations during training, and Q-Sparse applies top-k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). While each approach is effective in its own setting, it remains unclear how they compare under a common setup.

To address this gap, we pretrain Pythia-family models from scratch for controlled comparisons across model sizes (Biderman et al., 2023) to study the effects of (i) activation ℓ_1 pressure (P) and (ii) *thresholding* (T). While pressure adds a gradient component to induce smaller activation magnitudes, thresholding maps small values into exact zeros. Each one of them can be placed in different activation sites, giving a combinatorial design set of sparse interventions. For each com-

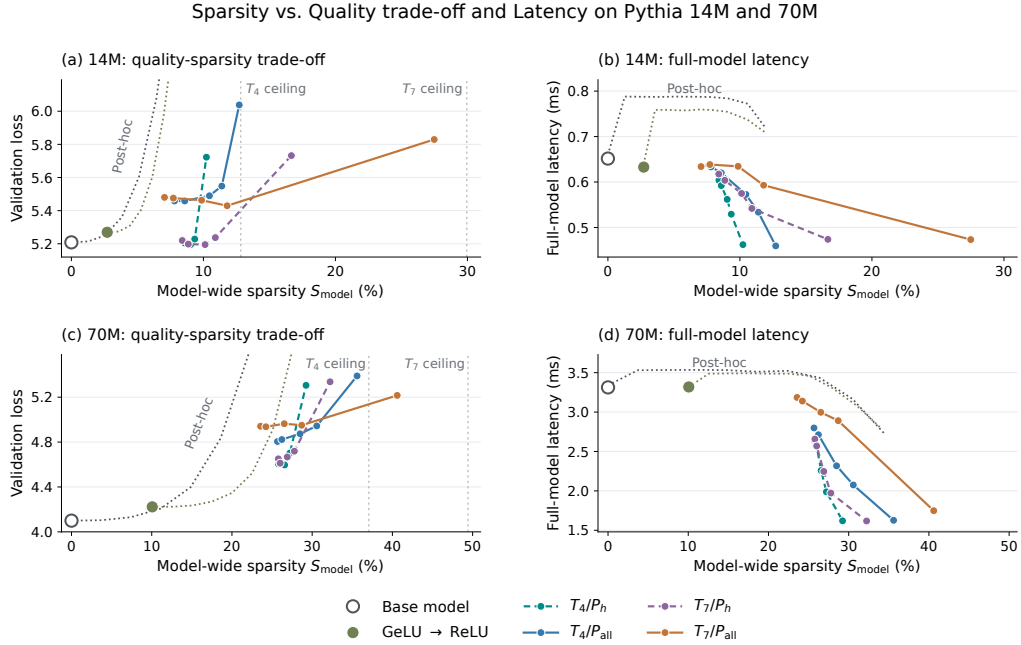


Figure 1: **Quality and execution depend on threshold and pressure placement.** Rows show Pythia-14M (a,b) and 70M (c,d); columns show final validation loss and full-model latency versus S_{model} . Each row contains 22 checkpoints: the Base Model (T_0/P_0), GeLU → ReLU at the hidden FFN activation (h) (T_1/P_0), and T_4/T_7 thresholding at four/seven sites with pressure applied either at h (P_h , dashed) or at all thresholded sites (P_{all} , solid). Figure 2 and Table 1 describe the intervention sites. The T/P curves connect separate runs with parameters controlling the degree of sparsity, while dotted post-hoc paths clip the fixed controls for $p = 0, .1, \dots, .9$. Vertical dashed lines mark the analytic sparsity ceilings.

bination of P and T , we derive an architecture-dependent maximum achievable sparsity ($S_{\text{model}}^{\text{max}}$), and measure model-wide attained activation sparsity (S_{model}) and the final inference latency under a specialized sparse kernel. Figure 1 presents sparsity vs. quality trade-offs and inference latency for 14M and 70M Pythia models, across P/T combinations and post-hoc clipping of baselines.

Across 36 14M-parameter and 22 70M-parameter training and evaluation runs, we identify several useful insights. (1) First, the T/P train-time interventions extend the attainable sparsity range relative to post-hoc clipping, approaching the architectural ceiling for the more aggressive recipes, but at a cost in quality. (2) Second, sparsity and speedup increase together within recipes; however, although they are correlated, a larger S_{model} does not directly translate into a larger speedup. The main requirement for lower latency is the location and structure of zeros and whether the kernel can exploit them, with most gains coming from skipped computation in FFN-hidden layers (h) and the attention output projection (z). (3) Third, for exploitable sparse structures, increasing model scale increases the opportunity for latency reductions. (4) Fourth, local FFN-hidden (h) pressure provides a more favorable balance between quality, sparsity and latency than global pressure when combined with broader thresholding. (5) Finally, local pressure can induce global changes in sparsity, showing that interventions targeted at individual activation sites can affect the model-wide sparsity structure. Taken together, these results indicate that activation sparsification should not be treated as an isolated training objective: training interventions and deployment kernels must be co-designed so that the induced sparsity is both high and structurally exploitable at inference time.

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes small activations at inference-time of a pre-trained model. While CATS thresholds small-magnitude activations within FFNs (Lee et al., 2024), TEAL extends it to attention (Liu et al., 2024). These studies motivate training-time interventions to adapt activation distributions to sparse computation and improve the trade-off between quality and sparsity.

ReLU Strikes Back studies the effect of activations functions in language models (Mirzadeh et al., 2023), showing clear sparsity gains on the use of ReLUs. ProSparse combines progressive ℓ_1 pressure and thresholding at FFNs during continued training (Song et al., 2024). It does not, however, extend its interventions to attention and restrict the analysis to local FFN sparsity. Cetin et al. (2026) extends ProSparse setup to pre-training and propose specialized kernels. These studies show that training can reshape the quality–sparsity trade-off, but they’re restricted to a single local intervention at FFN sites.

Q-Sparse extend sparsification beyond FFNs. It applies top- k selection to both attention and FFN projection inputs (Wang et al., 2024), but with no ℓ_1 pressure. Its top- k operation introduces, however, additional overhead due to the sorting operation. Spark Transformer tries to address it with statistical top- k (You et al., 2025), but we take a simpler approach and use a constant thresholding non-linearity. We argue that when the intervention is introduced during training, the network can adapt itself. Additionally, we evaluate the complementary effect of regularization pressure on both attention and FFNs.

2.2 INFERENCE SPEEDUP

Turning activation sparsity into speedup requires a specialized kernel adapted to the model architecture. TEAL accelerates batch-one autoregressive decoding, reporting up to $1.8\times$ speedup at 50% sparsity (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication with their TwELL kernel, and report $1.3\times$ speedup for batch inference. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory access must compesate the additional overhead introuced by the sparse operations. This development has been increasingly accelerated by the use of coding agents, through iterative kernel development with execution and profiling feedback (Ouyang et al., 2025; Luo et al., 2026). We take a related approach and use GPT-6 Astra do develop a specialized kernel based on our training interventions (OpenAI, 2026).

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

A sparsification intervention specifies where activations are thresholded (thresholding sites $\mathcal{N}$) and where activation pressure is applied (pressure sites $\mathcal{P}$). The sets $\mathcal{N}$ and $\mathcal{P}$ need not coincide, and the thresholding and pressure functions may take different forms. Together, these choices define a combinatorial space of training interventions. We restrict this space to keep the computational cost manageable while addressing the central research questions.

For thresholding, we avoid the overhead of exact top- k selection (Wang et al., 2024; You et al., 2025) and instead use one-sided and symmetric fixed-threshold operators $G_{+, \kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm, \kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. The constant κ controls the strength of the intervention and is held fixed within each training condition. At $\kappa = 0$, $G_{+, \kappa}$ reduces to ReLU, whereas $G_{\pm, \kappa}$ reduces to the identity.

For activation pressure, ℓ_1 averages across targeted sites and layers the mean absolute activation at each site after any thresholding. Ordinary L1 pressure minimizes $\mathcal{L}_{\text{task}} + \lambda\ell_1$, where λ controls the pressure intensity (Song et al., 2024; Cetin et al., 2026). To limit conflict between the ℓ_1 pressure and the task update while reducing the need to tune an additional hyperparameter, we introduce orthogonal ℓ_1 pressure (OL1), inspired by gradient-conflict projection methods (Yu et al., 2020; Hsieh et al., 2024). OL1 adds a separately preconditioned pressure correction after the task-only AdamW

update, projects away its component opposing the adaptive task direction, and caps its norm relative to that direction. At a fixed optimizer state, once the norm budget is active, further increasing λ leaves the pressure correction unchanged up to numerical stabilizers. To keep the multisite intervention study tractable, we use a common $\lambda = 1$ and budget $b = 1$ and directly measure whether the budget becomes active during training. Appendix A.1 gives the full formulation, a brief comparison and shows that budget saturation depends strongly on the target set.

Figure 2 locates the intervention sites within the Pythia architecture. The variables a and m denote the normalized inputs to the attention and FFN branches, respectively; h is the FFN hidden activation; z is the concatenated attention context before the output projection; and q , k , and v are the query, key, and value operands. Attention probabilities are denoted by p . We apply $G_{+, \kappa}$ at selected sites in $\{a, m, h, z\}$ and $G_{\pm, \kappa}$ at selected sites in $\{q, k, v\}$. When h is targeted, the thresholding nonlinearity replaces GELU.

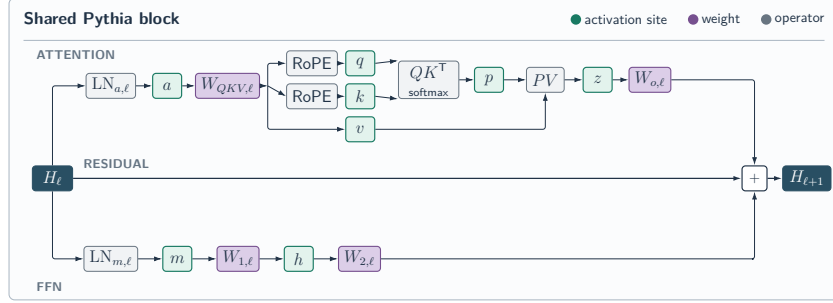


Figure 2: **Intervention sites in a shared Pythia block.** The attention and FFN branches use separate LayerNorms and join the residual path. Green nodes denote activation sites, and purple nodes denote learned weights. Table 1 specifies the thresholding and pressure assignments.

3.2 SPARSITY MEASURE AND ARCHITECTURAL REACH

Activation sparsity is the fraction of zero activations at a site. We define *model-wide sparsity* as $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in Pythia-block matrix products, and N_{model} counts all scalar multiplications in those blocks and the final output projection. We aggregate counts over validation data, excluding masked attention pairs.

To quantify intervention potential a priori, we define the *model-wide sparsity ceiling* as $\mathcal{S}_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$, where $N_{\text{reach}}(\mathcal{N})$ counts scalar multiplications guaranteed a zero activation operand if all selected thresholding sites are 100% sparse. Thus, $\mathcal{S}_{\text{model}}^{\text{max}}$ is the maximum sparsity directly attributable to $\mathcal{N}$. It depends on architecture, selected sites, and sequence length. Appendix B gives the precise accounting and definitions.

4 EXPERIMENTAL STUDY

We pretrain Pythia-14M, 70M, and 410M from random initialization (Biderman et al., 2023) on MiniPile (Kaddour, 2023), using 1.493 billion input tokens per model and training for one full-pass. We evaluate validation loss and $\mathcal{S}_{\text{model}}$ over all validation sequences. Table 1 defines the thresholding and pressure recipes and their sparsity ceilings across model sizes. It shows how the attainable sparsity ceiling increases with model size, under the same intervention recipe and fixed sequence length.

Experiments across model sizes. The 14M runs provide the most extensive controlled ablation, used to identify threshold and pressure regimes. The 70M runs test whether these regimes persist in a larger model. We also attempted to extend this analysis to 410M, but under the same token budget and number of optimization steps, this leaves the model substantially less trained. The baseline run with no intervention had a higher final loss and larger late-training gradient norm, which is evidence of a different training regime. We therefore treat the 410M result as a limited, partial scale-up, and focus the detailed intervention analysis on the 14M and 70M runs.

Within each model size and intervention, runs share initialization, data order and training budget. Appendix C.3 gives the full protocol. For comparison, we apply TEAL-style post-hoc magnitude thresholding (Liu et al., 2024) at a, m, h, z , with percentile targets $p = 0, 0.1, \dots, 0.9$ and site-layer thresholds calibrated separately on ten training blocks.

Table 1: Thresholding, pressure and sparsity ceilings. T_1 replaces GeLU with ReLU at h . Pressure uses OL1: $P_1 = P_h$ targets h , and P_{all} matches the thresholding sites ($\mathcal{P} = \mathcal{N}$). For T_4/T_7 , threshold strength is swept over $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$, with OL1 pressure weight $\lambda = 1$ and task-relative norm budget $b = 1$ fixed. Only the 14M T_1/P_1 ablation sweeps $\lambda \in \{0.05, 0.1, 0.5, 1\}$ at $b = 1$. Ceilings are $100 \cdot \mathcal{S}_{\text{model}}^{\text{max}}$ at 2,048 tokens; executed recipes by model size are listed in Appendix C.3.

Recipe	Threshold sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Ceiling (%)		
			14M	70M	410M
T_0/P_0 (Base Model)	$\emptyset$	$\emptyset$	0.00	0.00	0.00
T_1/P_0	h	$\emptyset$	4.28	12.35	24.93
T_1/P_1	h	h	4.28	12.35	24.93
T_4/P_0	a, m, h, z	$\emptyset$	12.83	37.06	74.78
T_4/P_h	a, m, h, z	h	12.83	37.06	74.78
T_4/P_{all}	a, m, h, z	$\mathcal{N}$	12.83	37.06	74.78
T_7/P_0	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
T_7/P_h	a, m, h, z, q, k, v	h	29.95	49.42	87.25
T_7/P_{all}	a, m, h, z, q, k, v	$\mathcal{N}$	29.95	49.42	87.25

Quality-sparsity trade-offs on Pythia 14M and 70M

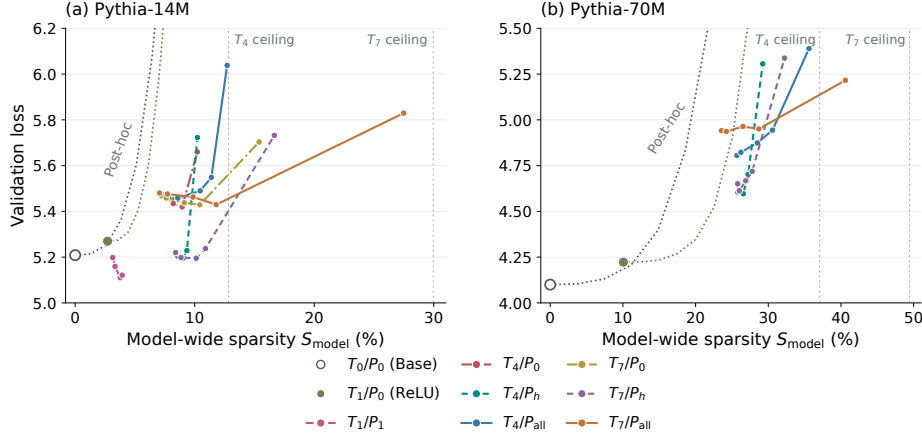


Figure 3: Quality-sparsity trade-offs at 14M and 70M. 36/22 trained conditions, with loss scales fitted separately to each size. Colors distinguish executed recipes; lines connect separate threshold settings (κ), or pressure weights (λ) for T_1/P_1 . Dotted paths apply post-hoc clipping at a, m, h, z to the fixed base and ReLU controls. Vertical guides mark the T_4 and T_7 analytic sparsity ceilings.

Train-time interventions extends the attainable sparsity range, close to the architectural ceiling for the more aggressive recipes, but at a quality cost. Figure 3 summarizes 58 trained conditions at 14M and 70M. At $\kappa = 0.5$, T_7/P_{all} reaches 27.48% and 40.60% model-wide sparsity: 91.8% and 82.2% of the respective ceilings. Validation loss increases by 0.62 and 1.12 relative to the same-size base model. At intermediate $\kappa \in 0.01, 0.05$, local pressure at h present a comparable sparsity at lower loss, compared to global pressure P_{all} . At Section 4.2 we do a comparative analysis on the effect of global vs. local pressure.

4.1 FROM SPARSITY TO SPEEDUPS

Agentic kernel development. To test whether learned sparsity reduces inference latency, we used Codex with GPT-6 Astra (OpenAI, 2026), to develop specialized GPU kernels under human guidance. We started from the kernels released by Cetin et al. (2026), although they required adaptation for Pythia architecture. The agent proposed, implemented and tested the kernels speed and accuracy. During the development phase, we sampled a subset from the training set and tested all kernel changes on them, comparing the model’s output without the custom GPU implementation. We rejected kernels that provided a different model output response. Once we found a kernel that provided speedup for a single sparse model with the same response, we tested it for each training recipe. For each model, we measured inference time and output correctness over the validation set. Its speedup was recorded and included only if these checks passed. We then repeated this evaluation at 70M after adapting the kernels to its larger matrix dimensions. Although we are not specialized in GPU-kernel design, this workflow made direct execution tests practical. This case illustrates how coding agents can support systems development.

Sparsity and speedup increase together within recipes. For the implemented kernel, Figure 4 shows an approximately linear increase within each trained threshold/pressure family over the tested grid, although with different slopes across families. At 14M and $\kappa = 0.5$, for example, T_7/P_{all} is much sparser than T_4/P_{all} (27.48% versus 12.71%), yet slightly slower ($1.38\times$ versus $1.42\times$ base-model speedup). Although they correlate, a larger $\mathcal{S}_{\text{model}}$ does not directly translate into a larger speedup. The main requirement is the place and structure of zeros and wheater the kernel can exploit them. We investigate it on Section 4.3.

For exploitable structure, scale increases the opportunity. Figure 4 shows that the attainable full-model speedup increases with model size. Larger models combine larger matrix operations with higher sparsity ceilings, making sparse computation more effective at reducing end-to-end latency. Accordingly, the best observed speedup rises from $1.42\times$ at 14M to $2.08\times$ at 70M.

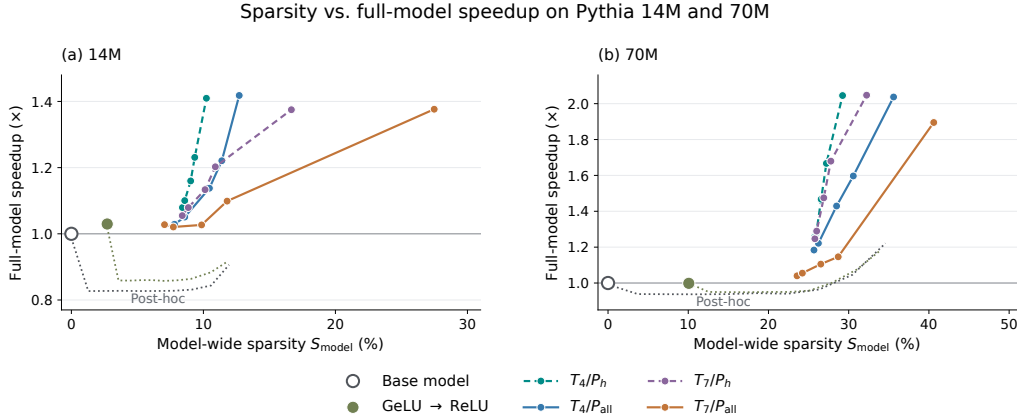


Figure 4: **From model-wide sparsity to measured speedup.** Pythia-14M (a) and 70M (b), with 22 trained checkpoints and 20 post-hoc clipping settings per size. Speedup divides the same-size base-model latency by each setting’s latency, using the final specialized implementation for both terms. Timings use RTX5090, BF16, batch one, 2,048 tokens and the full logit head. Sparsity uses full-validation pooled logical counts.

Post-hoc clipping does not yield comparable gains in this implementation. Every post-hoc clipping point remains below $1\times$ at 14M, and the first degradation point show the additional overhead cost. At 70M, aggressive clipping can exceed $1\times$, reaching $1.22\times$, but with substantial loss degradation. Here clipping uses a separate masking operations around the kernel, adding additional overhead because it has to iterate over activations on every forward pass. The resulting zeros still have to form groups that the kernel skipping rules can use. The overhead can outweigh the avoided products. However, we did not optimize a fused clipping implementation, so this result does not establish a general limitation of post-hoc sparsification, but **points into a requirement of pairing the sparse intervention with specialized implementation.**

4.2 PAIRED EFFECTS OF GLOBAL VS. LOCAL PRESSURE

One question raised during sparse intervention design is wheater **activation pressure should be applied over the entire network, or are there local sites more suceptible to sparsification?** To answer this, we compare thresholding across multiple sites (T_4 and T_7) with global and local FFN hidden pressure (P_{all} and P_h) at Figure 5.

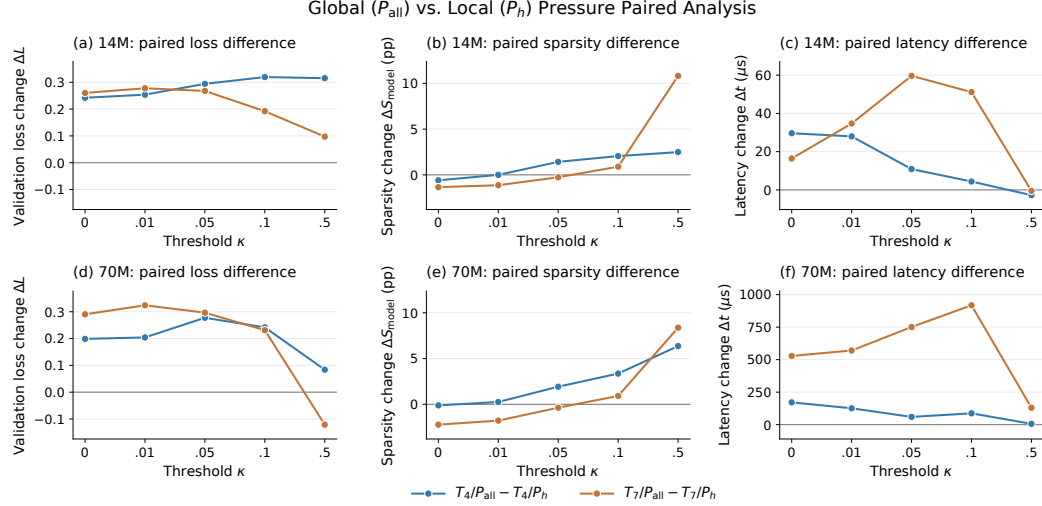


Figure 5: **Paired effects of broadening pressure from P_h to P_{all} .** Every point is P_{all} minus P_h at matched model size, threshold sites and κ , with $\lambda = b = 1$. Rows show 14M/70M; columns show loss change, sparsity change (pp), and full-model latency change (μs). Positive loss or latency differences favor P_h ; positive sparsity differences favor P_{all} .

Global pressure achieves higher sparsity at a greater quality cost, without translating this additional sparsity into lower latency. At moderate thresholds ($\kappa \leq 0.1$), local h-only pressure (P_h) provides a more favorable quality–sparsity trade-off than global pressure (P_{all}) in both model sizes: P_h lowers loss in all 16 matched comparisons at 14M and 70M (Figure 5), with reductions of 0.19–0.32 while sparsity differs by at most 3.36 percentage points. Although broader pressure can achieve higher sparsity, this gain does not translate into lower measured latency: h-only pressure is faster in 18 of the 20 matched pairs, including all ten 70M pairs. Thus, applying sparsity pressure globally trades model quality for additional sparsity without yielding a corresponding inference-time benefit.

Local pressure can produce nonlocal sparsity changes. Figure 6 present the sparsity by site and layer accross T_4 and T_7 interventions, for all pressures at 14M. At $\kappa = 0.05$, P_h increases sparsity at both h and z relative to P_0 , although z receives no direct pressure. Under T_7 , P_h also produces higher sparsity at h and z than P_{all} , despite the latter targeting both sites. One possible explanation is that (q,k,v) increases ℓ_1 pressure norms, and P_{all} caps the OL1 norm more frequently (Appendix A.1), reducing the local at h . The takeaway is that a local site pressure may spillover and affect the distributions of other sites.

(h, z) are more suceptible to sparsification. At higher threshold ($\kappa = 0.5$) (h,z) sites become fully sparse, even if no pressure is applied; while (q,k,v) sites are in the $[14, 36]$ and (a,m) are in $[57, 70]$ sparsity range. This implies the distribution of (q,k,v) and (a,m) may have different ranges and require different threshold values than (h,z).

4.3 WHEN SPARSITY TRANSLATES TO SPEEDUP

Sparsity reduces latency when avoided work exceeds the overhead cost of detecting and skipping it. At h, z , the sparse kernel checks 8×16 activation tiles before loading their associated weights, so empty tiles avoid both weight load from memory and the matrix multiplications. This strategy, however, doesn’t extend to other sites. Applying it at a and m increased full-model latency by 17.2%

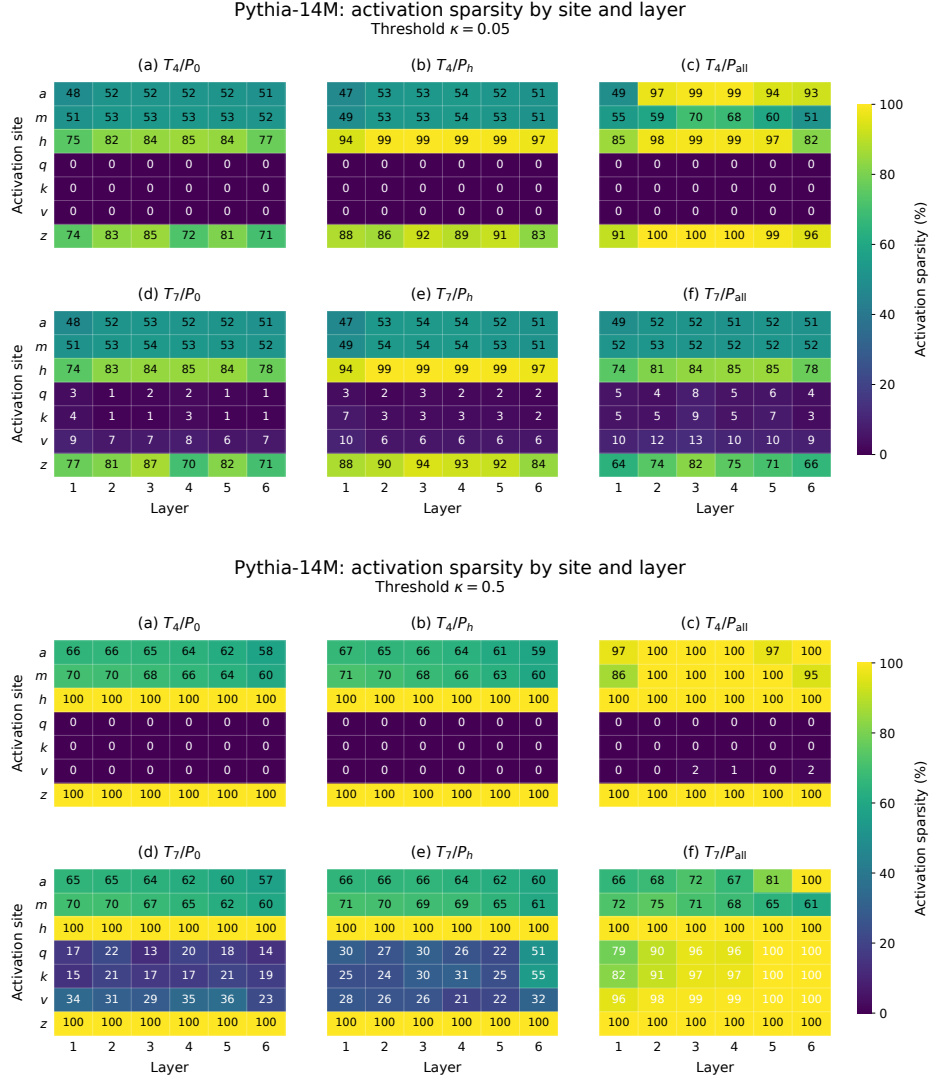


Figure 6: Activation sparsity by site and layer in Pythia-14M. The six threshold/pressure recipes at $\kappa = 0.05$ (top) and 0.5 (bottom). Each cell gives the percentage of zero activations over all complete validation sequences at one site and layer. Q/K are post-RoPE; all panels share a 0–100% color scale. Values are rounded to whole percentages, so 0 or 100 does not establish a completely dense or all-zero tensor.

and 27.4% on 14M T_7/P_{all} at $\kappa = 0.5$, with unchanged outputs. Only 11.12% of a 's 8×16 tiles were empty, and none at m , so most groups still required matrix multiplication and the additional overhead was not justified.

Attention follows FlashAttention's tiled sequence (Dao et al., 2022): load q, k , compute scores, apply masking and softmax, then multiply probabilities by v . Our implementation add additional overhead after the checks, so skipping an empty tile's multiplication cannot recover the latency from reading from memory. Softmax still runs and turns zero scores into nonzero probabilities, so q/k sparsity does not automatically propagate to pv . Zeros in v remain usable, and sparsifying the probabilities is a further untested opportunity.

We isolate each site's effect at 14M by disabling it during inference time. Table 2 reports the *bypass* (i.e. percentage of skipped matrix instructions) and saved time per-site sparse operation for the sparser 14M recipe T_7/P_{all} , $\kappa = 0.5$. It makes clear that savings come from h (150 μs) and z (29 μs). Individual a, m, q, k effects remain unresolved against process variation, whereas pv skipping adds latency. Enabling qk/pv skipping together raises full-model latency from 0.514 to 0.521 ms despite substantial bypass. Thus, for this checkpoint and implementation, attention's arithmetic savings do not offset its overhead, while h/z benefit from avoiding weight reads as well as computation.

Table 2: **Conditional latency savings:** 14M T_7/P_{all} , $\kappa = 0.5$. Latency is measured per 2,048-token sequence on RTX 5090 (BF16, batch one). Saved time is full-model latency with one skipping path off minus latency with all paths on; positive values indicate a benefit (reduced latency). Brackets give [minimum, maximum] savings across all off/on pairs of three independent timing runs. Each run reports the geometric mean over 64 sequences.

Operation	Bypass (%)	Saved time (μs)	Process span (μs)
a : QKV projection	8.4	-1.6	[-6.0, +3.8]
m : FFN-up	0.0	-2.4	[-5.1, +2.7]
h : FFN-down	94.5	+150.1	[+145.4, +154.4]
z : attention-output projection	99.6	+29.0	[+23.7, +35.7]
q, k : QK scores	58.0	-2.9	[-7.3, +1.1]
v : PV product	67.2	-6.2	[-9.0, -2.2]

5 DISCUSSION AND CONCLUSION

Our results suggest that sparsification should be understood as a joint training-and-deployment problem rather than as a property of activation distributions alone. First, **local pressure does not imply a local response**: applying pressure only at the FFN hidden site can alter sparsity at other sites, showing that the network can reorganize its internal representations in response to a localized training intervention. More broadly, **sparsity is useful only if the resulting zero structure can be exploited by the inference system**. S_{model} is best viewed as a training-time proxy for computational opportunity, not as a direct predictor of latency. In our measurements, the latency gains comes from (h, z) sites, which were the ones more susceptible to sparsification, with higher achieved sparsity. One hypothesis is that we must achieve a high ($> 90\%$) level of site-layer sparsity, so the zeros gets a exploitable tiled structure with higher probability.

Transformers appear sufficiently flexible to adapt their internal representations to architectural and optimization constraints, suggesting that larger gains may come from **designing training objectives around inference requirements**, rather than first maximizing a generic sparsity metric and only afterward attempting to accelerate it. The same principle may extend beyond activation sparsity: training could be used to shape representations for properties required by weight sparsity, quantization, batched execution, or other hardware-specific constraints. In this view, deployment requirements are not merely constraints imposed after training, but potential signals for deciding what properties the network should learn.

Several limitations define the scope of these conclusions and point to concrete directions for further work. Our scaling evidence is limited: although the 410M experiments demonstrate that high model-wide sparsity remains attainable, the fixed token budget leaves 410M in a substantially different optimization regime, so it should be interpreted as a stress test rather than a clean scaling experiment.

Our runtime study is also restricted to batch-one, full-sequence inference on an RTX5090; we do not study how sparsity interacts with batched inference, cached decoding, or other hardware.

In addition, the full intervention space is combinatorial, and under the available compute budget we evaluate only a selected set of threshold and pressure placements rather than an exhaustive factorial study. The nonlocal response visible in Figure 6 suggests one particularly promising direction: interventions concentrated at h and z may preserve much of the execution benefit while avoiding pressure or thresholding at sites whose sparsity is expensive in quality or unprofitable for the current kernel. Related extensions include learning or tuning different thresholds κ at different sites and directly sparsifying attention probabilities after softmax, which could expose work in pv without relying on q/k zeros to propagate through softmax. More generally, rather than asking where the model can be made sparse, a productive systems-guided approach is to decompose full-model inference into individual operations, identify where latency and removable work are concentrated, determine what structured representation would allow those operations to be skipped efficiently, and then design training-time interventions that encourage precisely that structure.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.

- OpenAI. GPT-6 Astra: A New Generation of Intelligence. OpenAI, September 2026. URL <https://openai.com/index/gpt-6-astra/>. Accessed September 19, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 ACTIVATION PRESSURE AND OL1

Objective and update. For J activation tensors A_j at the sites and layers targeted by $\mathcal{P}$, each with n_j entries after any thresholding, define

$$\ell_1(\theta; \mathcal{P}) = \frac{1}{J} \sum_{j=1}^J \frac{1}{n_j} \sum_{i=1}^{n_j} |A_{j,i}|. \quad (1)$$

Each tensor has equal weight, so changing $\mathcal{P}$ changes both the targets and normalization. Ordinary L1 applies AdamW (Loshchilov & Hutter, 2019) to the globally clipped gradient of $\mathcal{L}_{\text{task}} + \lambda \ell_1$. OL1 separates task and pressure updates: task-only AdamW is followed by a pressure correction that removes the opposing component and limits its norm. Pressure therefore does not directly enter Adam’s moment estimates.

Both gradients are evaluated at the same parameters and averaged over the same microbatches. The clipped task gradient updates AdamW’s bias-corrected moments $\hat{m}_t, \hat{v}_t$; the pressure gradient $g_1 = \nabla_{\theta} \ell_1$ is unclipped and has no momentum. Using the same preconditioner for both directions gives

$$u = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon_A}, \quad w = \frac{g_1}{\sqrt{\hat{v}_t} + \epsilon_A}. \quad (2)$$

Norms and inner products include parameters with both gradients and initialized Adam state. For $c = \langle u, w \rangle$, $q = \|u\|_2^2$ and stabilizer $\epsilon > 0$,

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (3)$$

Aligned pressure is retained. For weight $\lambda \geq 0$ and relative norm budget $b > 0$,

$$r = \frac{\lambda \|\tilde{w}\|_2}{\|u\|_2 + \epsilon}, \quad s = \min \left\{ 1, \frac{b}{r + \epsilon} \right\} \quad (r > 0), \quad (4)$$

with $s = 1$ when $r = 0$. After task-only AdamW, each parameter group p receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$, where α_p is its learning rate. Before learning-rate scaling and ignoring stabilizers, the correction norm is $\min\{\lambda \|\tilde{w}\|_2, b \|u\|_2\}$. At a fixed optimizer state, increasing λ has no further effect once the cap is active.

OL1 combines conflict projection (Yu et al., 2020) and a smoothed task direction (Hsieh et al., 2024) in Adam’s preconditioned coordinates. The stabilizer makes orthogonality approximate. This calculation excludes group learning rates and decoupled weight decay; avoiding opposition to the Adam direction does not guarantee unchanged task or validation loss.

L1 versus OL1. At 14M T_1/P_1 , matched initialization, data order and training budget yield similar results (Table 3). Sparsity differs by less than 0.02 percentage points at every λ . OL1 has slightly lower loss at $\lambda \in \{0.05, 0.1, 0.5\}$ and higher loss at $\lambda = 1$. One seed cannot establish whether these small loss differences persist across seeds.

Table 3: **L1 and OL1 at T_1/P_1 on Pythia-14M.** Final validation loss and model-wide sparsity after 712 matched training steps; OL1 uses $b = 1$. Both metrics cover all 338 complete validation blocks (500 documents; 1,444 tail tokens excluded). One initialization per condition.

λ	L1		OL1	
	S_{model} (%)	Val. loss	S_{model} (%)	Val. loss
0.05	3.1416	5.2061	3.1452	5.1981
0.1	3.3363	5.1655	3.3386	5.1594
0.5	3.7876	5.1127	3.7680	5.1102
1	3.9493	5.1023	3.9384	5.1210

Update geometry and budget saturation. We measure $\rho_{\text{opp}} = \max(0, -c)/(q + \epsilon)$, the removed component’s norm relative to $\|u\|_2$ before capping. All recorded steps satisfy $q > \epsilon$; aligned steps contribute zero. Figure 7 shows ρ_{opp} and r/b for the 20 14M OL1 conditions in Figure 1, using all 712 updates per condition at $\lambda = b = 1$. The cap is active ($s < 1$) on 96.99% of T_7/P_{all} steps, compared with 0.51% for T_4/P_{all} and 0.25% for either P_h recipe. The diagnostics are consistent with stronger pressure–task conflict when attention sites are included, but do not isolate each site’s contribution.

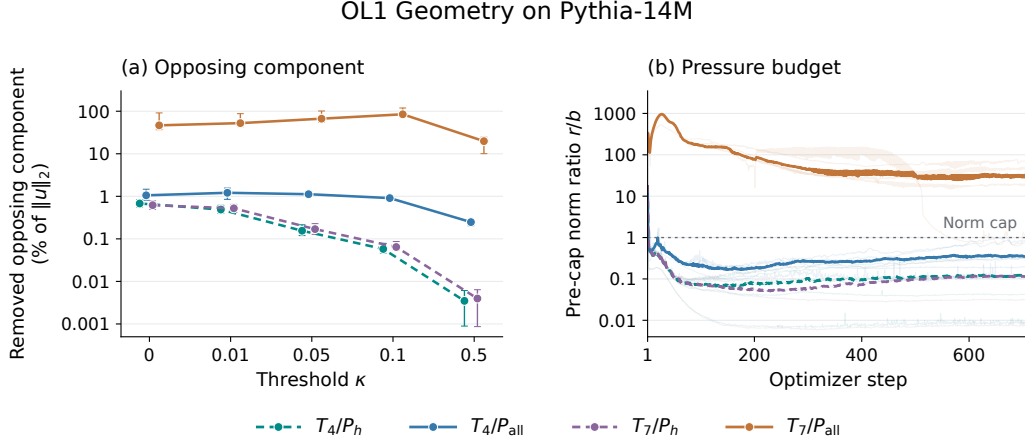


Figure 7: **OL1 geometry depends on pressure placement at 14M.** (a) Median removed opposing component, $100\rho_{\text{opp}}$; bars span the 25th–75th percentiles over 712 training steps per condition, not confidence intervals. (b) Pre-cap norm ratio r/b : faint curves show individual thresholds, and bold curves their pointwise median. The dotted line marks the norm cap. Colors and line styles match Figure 1; both y-axes are logarithmic, and $\lambda = b = 1$ throughout.

B SPARSITY ACCOUNTING

B.1 ACTIVATION AND MODEL-WIDE SPARSITY

Activation sparsity is the fraction of zero entries at a site, pooling counts across layers and validation sequences before dividing.

Model-wide sparsity counts scalar multiplications with a zero activation input:

$$\mathcal{S}_{\text{model}} = \frac{N_0}{N_{\text{model}}}. \quad (5)$$

N_0 counts these multiplications within Pythia blocks; N_{model} counts all multiplications in those blocks and the final vocabulary projection. The latter contributes only to the denominator. Counts are pooled across layers and validation sequences; percentages are $100\mathcal{S}_{\text{model}}$.

For a block’s linear projection from d_{in} to d_{out} features over n rows, the denominator contribution is $nd_{\text{in}}d_{\text{out}}$. Each zero input contributes d_{out} to N_0 , one multiplication per output feature. Zero weights do not count as activation sparsity.

In attention, both operands are activations. We count each multiplication once if either or both are zero. For queries q , keys k , values v and attention probabilities p , one head in one sequence contributes

$$N_{0,\text{QK}} = \sum_{t=1}^T \sum_{u=1}^t \sum_{j=1}^{d_h} \mathbf{1}\{q_{tj} = 0 \vee k_{uj} = 0\}, \quad (6)$$

$$N_{0,\text{PV}} = \sum_{t=1}^T \sum_{u=1}^t \sum_{j=1}^{d_h} \mathbf{1}\{p_{tu} = 0 \vee v_{uj} = 0\}. \quad (7)$$

Here T is sequence length, d_h the head dimension, t the query position and u the key/value position. Restricting $u \leq t$ excludes masked future positions from both counts. We sum over heads, layers and validation sequences, using the actual operands after thresholding and intervening transformations.

Normalization, softmax, activation functions, additions and memory transfers are excluded, so $\mathcal{S}_{\text{model}}$ does not directly measure latency or speedup.

B.2 MODEL-WIDE SPARSITY CEILING

Assume all activations at the thresholding sites $\mathcal{N}$ are zero. If $N_{\text{reach}}(\mathcal{N})$ counts the multiplications that must then have a zero activation input, the ceiling is

$$\mathcal{S}_{\text{model}}^{\text{max}} = \frac{N_{\text{reach}}(\mathcal{N})}{N_{\text{model}}}. \quad (8)$$

The denominator is unchanged, and each multiplication is counted once. The numerator includes downstream activations forced to zero: setting $v = 0$ also makes $z = 0$, affecting both PV and the attention-output projection. Zero attention scores do not imply zero softmax probabilities.

The ceiling depends only on architecture, selected sites and sequence length. Natural zeros outside those sites can raise measured $\mathcal{S}_{\text{model}}$ above it; the ceiling does not predict the associated quality cost. Appendix C.2 gives the counts for each Pythia size.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA ARCHITECTURE AND INTERVENTION SITES

Table 4 gives the shapes and positions of the sites in Figure 2. Here T is sequence length, d hidden width, d_f FFN width and H the number of heads, each with dimension $d_h = d/H$. Sites and threshold are shared across layers.

Table 4: Activation shapes and intervention positions for one sequence.

Site	Shape	Following matrix product	Position
a	$T \times d$	QKV projection	After attention LayerNorm
m	$T \times d$	FFN up projection	After FFN LayerNorm
h	$T \times d_f$	FFN down projection	Activation-function output
q, k	$H \times T \times d_h$	Attention scores	After RoPE
v	$H \times T \times d_h$	Probability-value	After the QKV split
z	$T \times d$	Attention-output projection	Concatenated attention context

C.2 ARCHITECTURE-SPECIFIC SPARSITY CEILINGS

For one sequence, each Pythia block uses $3Td^2$ scalar multiplications in the QKV projection, Td^2 in the attention-output projection, Tdd_f in each FFN projection, and $dT(T+1)/2$ in each of QK and PV . With L layers and vocabulary size V_{vocab} , the totals are

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (9)$$

Sequence count cancels from the ceiling ratio. For $d_f = 4d$, per-layer counts in the ceiling numerator are 0 for T_0/P_0 , $4Td^2$ for T_1/P_0 and T_1/P_1 , $12Td^2$ for T_4 , and C_{block} for T_7 . Multiplying by L/C_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$, independent of pressure placement.

Table 1 uses $V_{\text{vocab}} = 50,304$ and $(L, d) = (6, 128), (6, 512), (24, 1,024)$ for 14M, 70M and 410M, respectively, with the common sequence length in Table 5.¹

¹Pinned architecture configurations: 14M, 70M, and 410M.

C.3 TRAINING AND EVALUATION

All sizes use the Pythia-14M-deduped tokenizer and seeds of 1234 for initialization and data order, with one seed per condition. AdamW settings are shared: $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and gradient clipping at norm 1. Attention and hidden dropout are zero. Training uses 712 updates with 1% linear warmup followed by cosine learning-rate decay. Table 5 gives the learning rates, microbatch sizes and shared effective batch. Gradients accumulate over the listed microbatches before each optimizer update.

Table 5: **Training settings across model sizes.** Effective batch size equals microbatch size times gradient accumulation. The common training budget gives fewer tokens per parameter at larger sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation (microbatches)	32	256	256
Effective batch (sequences/update)	1,024	1,024	1,024
Sequence length	2,048	2,048	2,048
Tokens per parameter	106.14	21.20	3.68

All sizes include the T_0/P_0 and T_1/P_0 controls. The 14M study adds T_1/P_1 with L1 or OL1, and T_4 and T_7 with P_0 , P_h or P_{all} . At 70M, T_4 and T_7 use P_h or P_{all} ; at 410M, only P_{all} . Parameter grids are given in Table 1. We evaluate the final checkpoint, without validation-based selection. The main figures measure loss and model-wide sparsity on the same validation sequences (Appendix B).

Figure 8 shows the training trajectories. They do not establish convergence, and the lower learning rates at 410M limit comparisons that attribute loss differences to model size alone.

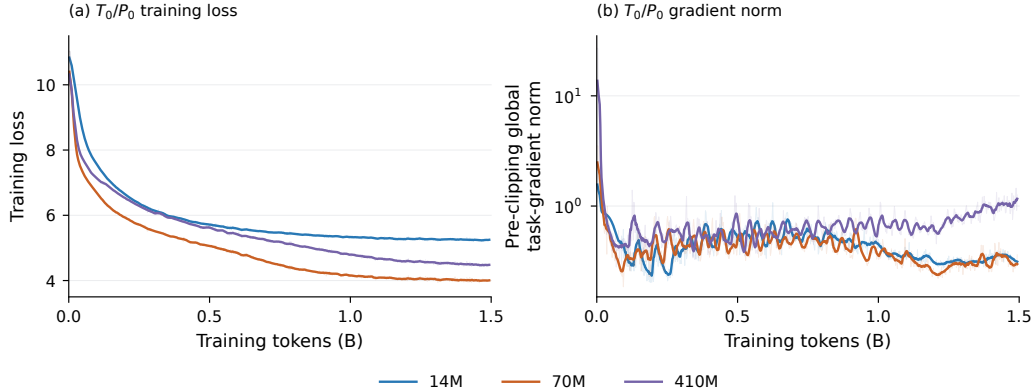


Figure 8: **Base-model (T_0/P_0) training across model sizes.** (a) Training loss and (b) gradient norm before clipping (logarithmic scale). Faint lines show individual updates; solid lines show centered nine-update means, with shorter windows at the ends. Gradient norms are computed after accumulation and are not normalized by parameter count.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

The following tables report the original 64 trained endpoints and the ten additional 70M h-only endpoints, the 29 original paired 14M contrasts, 25 additional pressure-scope contrasts, and all 15 cross-scale broad-pressure recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are

formed from pooled integer product counts. The endpoint table uses the common seven-site sparsity-ceiling denominator from Table 1, rather than a different denominator for each recipe. Here $U_{\text{arch}} = \mathcal{S}_{\text{model}}/\mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$ denotes this common-reference ratio, retained for the complete endpoint table.

Table 6: The original 64 trained endpoints; Table 7 adds the ten 70M h-only conditions. T_i/P_j specifies threshold and pressure scope; P_0 means no pressure. U_{arch} uses the common seven-site ceiling within each size. H-only losses use ordinary final validation; other losses use the logical diagnostic pass (Appendix C.3). Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	T_4/P_0	$\kappa = 0$	5.4705	7.212	24.078
14M	T_4/P_0	$\kappa = 0.01$	5.4665	7.414	24.752
14M	T_4/P_0	$\kappa = 0.05$	5.4341	8.206	27.396
14M	T_4/P_0	$\kappa = 0.1$	5.4197	8.953	29.891
14M	T_4/P_0	$\kappa = 0.5$	5.6597	10.216	34.106
14M	T_4/P_h	$\kappa = 0$	5.2157	8.409	28.076
14M	T_4/P_h	$\kappa = 0.01$	5.2045	8.586	28.665
14M	T_4/P_h	$\kappa = 0.05$	5.1956	9.036	30.167
14M	T_4/P_h	$\kappa = 0.1$	5.2287	9.343	31.194
14M	T_4/P_h	$\kappa = 0.5$	5.7227	10.227	34.146
14M	T_4/P_4	$\kappa = 0$	5.4583	7.810	26.075
14M	T_4/P_4	$\kappa = 0.01$	5.4583	8.587	28.668
14M	T_4/P_4	$\kappa = 0.05$	5.4898	10.456	34.910
14M	T_4/P_4	$\kappa = 0.1$	5.5483	11.394	38.041
14M	T_4/P_4	$\kappa = 0.5$	6.0380	12.713	42.446
14M	T_7/P_0	$\kappa = 0$	5.4684	7.218	24.097
14M	T_7/P_0	$\kappa = 0.01$	5.4588	7.618	25.433
14M	T_7/P_0	$\kappa = 0.05$	5.4379	9.127	30.471
14M	T_7/P_0	$\kappa = 0.1$	5.4287	10.425	34.805
14M	T_7/P_0	$\kappa = 0.5$	5.7029	15.387	51.371
14M	T_7/P_h	$\kappa = 0$	5.2198	8.399	28.042
14M	T_7/P_h	$\kappa = 0.01$	5.1981	8.856	29.567
14M	T_7/P_h	$\kappa = 0.05$	5.1950	10.126	33.807
14M	T_7/P_h	$\kappa = 0.1$	5.2374	10.904	36.403
14M	T_7/P_h	$\kappa = 0.5$	5.7320	16.664	55.634
14M	T_7/P_7	$\kappa = 0$	5.4802	7.054	23.551
14M	T_7/P_7	$\kappa = 0.01$	5.4758	7.723	25.786
14M	T_7/P_7	$\kappa = 0.05$	5.4628	9.863	32.929
14M	T_7/P_7	$\kappa = 0.1$	5.4295	11.797	39.385
14M	T_7/P_7	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	T_4/P_4	$\kappa = 0$	4.8054	25.673	51.944
70M	T_4/P_4	$\kappa = 0.01$	4.8229	26.224	53.060
70M	T_4/P_4	$\kappa = 0.05$	4.8742	28.490	57.644
70M	T_4/P_4	$\kappa = 0.1$	4.9439	30.574	61.861
70M	T_4/P_4	$\kappa = 0.5$	5.3895	35.596	72.022
70M	T_7/P_7	$\kappa = 0$	4.9412	23.562	47.674
70M	T_7/P_7	$\kappa = 0.01$	4.9364	24.226	49.017
70M	T_7/P_7	$\kappa = 0.05$	4.9637	26.524	53.666
70M	T_7/P_7	$\kappa = 0.1$	4.9501	28.694	58.056
70M	T_7/P_7	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	T_4/P_4	$\kappa = 0$	5.6921	38.339	43.944

Table 6 (continued)

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
410M	T_4/P_4	$\kappa = 0.01$	5.6914	39.149	44.873
410M	T_4/P_4	$\kappa = 0.05$	5.6789	42.125	48.283
410M	T_4/P_4	$\kappa = 0.1$	5.6789	45.688	52.367
410M	T_4/P_4	$\kappa = 0.5$	5.1910	71.591	82.058
410M	T_7/P_7	$\kappa = 0$	5.4263	41.121	47.133
410M	T_7/P_7	$\kappa = 0.01$	5.4331	41.478	47.542
410M	T_7/P_7	$\kappa = 0.05$	5.4679	44.053	50.494
410M	T_7/P_7	$\kappa = 0.1$	5.4907	48.998	56.161
410M	T_7/P_7	$\kappa = 0.5$	5.1207	80.616	92.401

Table 7: Additional 70M h-only endpoints in Figure 1. Loss uses ordinary reloaded final validation; sparsity uses canonical FP16 logical counts. U_{arch} uses the common seven-site ceiling, as in Table 6. Together the two tables cover all 74 trained conditions.

Recipe	κ	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
T_4/P_h	0	4.6065	25.792	52.185
T_4/P_h	0.01	4.6185	25.971	52.547
T_4/P_h	0.05	4.5964	26.566	53.752
T_4/P_h	0.1	4.7020	27.218	55.071
T_4/P_h	0.5	5.3057	29.236	59.153
T_7/P_h	0	4.6508	25.786	52.173
T_7/P_h	0.01	4.6122	26.009	52.624
T_7/P_h	0.05	4.6670	26.895	54.416
T_7/P_h	0.1	4.7189	27.779	56.206
T_7/P_h	0.5	5.3372	32.233	65.216

D.1 ACTIVATION SPARSITY AND SIGNED-DENSITY COVERAGE

The seven frozen-checkpoint evaluations in Figure 11 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 11 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention in Figure 11. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%. These historical densities cover no h-only multisite checkpoints. Figures 9 and 10 use retained final-validation site counters for all six threshold/pressure combinations, separating m and h . These counts were measured independently of the signed histogram reruns; small pass-to-pass rounding differences remain visible. The retained logical-product counters separately decompose model-wide sparsity into operation contributions at $\kappa = 0.05$ and 0.5.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional thresholding. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 8: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1(all) – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1(all) – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1(all) – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1(all) – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1(all) – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1(all) – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1(all) – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1(all) – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1(all) – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1(all) – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 9: Pressure-scope increments and threshold-placement contrasts at 14M. Each row subtracts the reference at the same κ . These are single-seed comparisons.

Treatment – reference	κ	ΔL	$\Delta\mathcal{S}_{\text{model}}$ (pp)
$T_4/P_h - T_4/P_0$	0	-0.2548	+1.197
$T_4/P_h - T_4/P_0$	0.01	-0.2620	+1.172
$T_4/P_h - T_4/P_0$	0.05	-0.2385	+0.830
$T_4/P_h - T_4/P_0$	0.1	-0.1910	+0.390
$T_4/P_h - T_4/P_0$	0.5	+0.0630	+0.012
$T_4/P_4 - T_4/P_h$	0	+0.2425	-0.599
$T_4/P_4 - T_4/P_h$	0.01	+0.2538	+0.001
$T_4/P_4 - T_4/P_h$	0.05	+0.2942	+1.421
$T_4/P_4 - T_4/P_h$	0.1	+0.3196	+2.051
$T_4/P_4 - T_4/P_h$	0.5	+0.3153	+2.486
$T_7/P_h - T_7/P_0$	0	-0.2486	+1.182
$T_7/P_h - T_7/P_0$	0.01	-0.2607	+1.238
$T_7/P_h - T_7/P_0$	0.05	-0.2429	+0.999
$T_7/P_h - T_7/P_0$	0.1	-0.1913	+0.479
$T_7/P_h - T_7/P_0$	0.5	+0.0292	+1.277
$T_7/P_7 - T_7/P_h$	0	+0.2604	-1.345
$T_7/P_7 - T_7/P_h$	0.01	+0.2777	-1.133
$T_7/P_7 - T_7/P_h$	0.05	+0.2678	-0.263
$T_7/P_7 - T_7/P_h$	0.1	+0.1921	+0.893
$T_7/P_7 - T_7/P_h$	0.5	+0.0974	+10.819
$T_7/P_h - T_4/P_h$	0	+0.0041	-0.010
$T_7/P_h - T_4/P_h$	0.01	-0.0064	+0.270
$T_7/P_h - T_4/P_h$	0.05	-0.0006	+1.090
$T_7/P_h - T_4/P_h$	0.1	+0.0087	+1.560
$T_7/P_h - T_4/P_h$	0.5	+0.0094	+6.436

Table 10: All matched A7-OL1(all) minus A4-OL1(all) comparisons. Negative loss changes and positive sparsity changes favor A7-OL1(all).

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

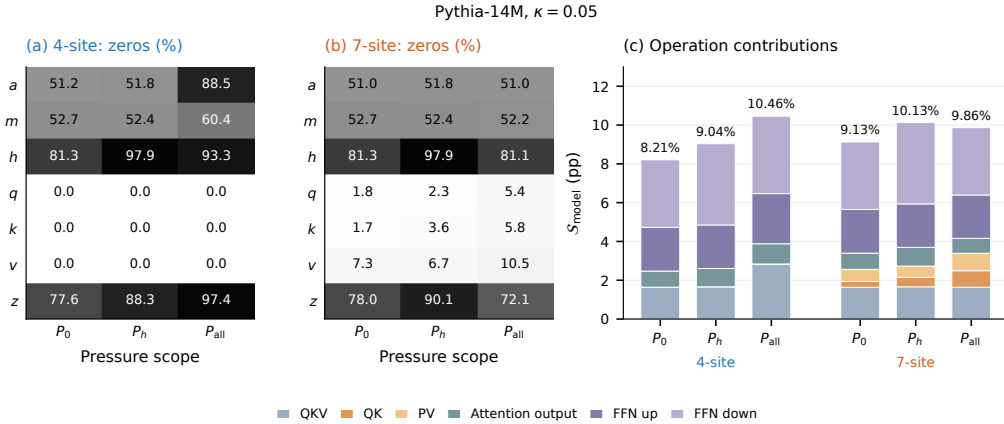


Figure 9: **Pressure-scope effects at the moderate threshold $\kappa = 0.05$.** The six 14M recipes use pooled counts and the full-model operation denominator defined in Appendix B. H-only pressure concentrates zeros at h while preserving a different pattern at m and Q/K/V from broad pressure. Rounded percentages do not establish all-zero or fully dense tensors.

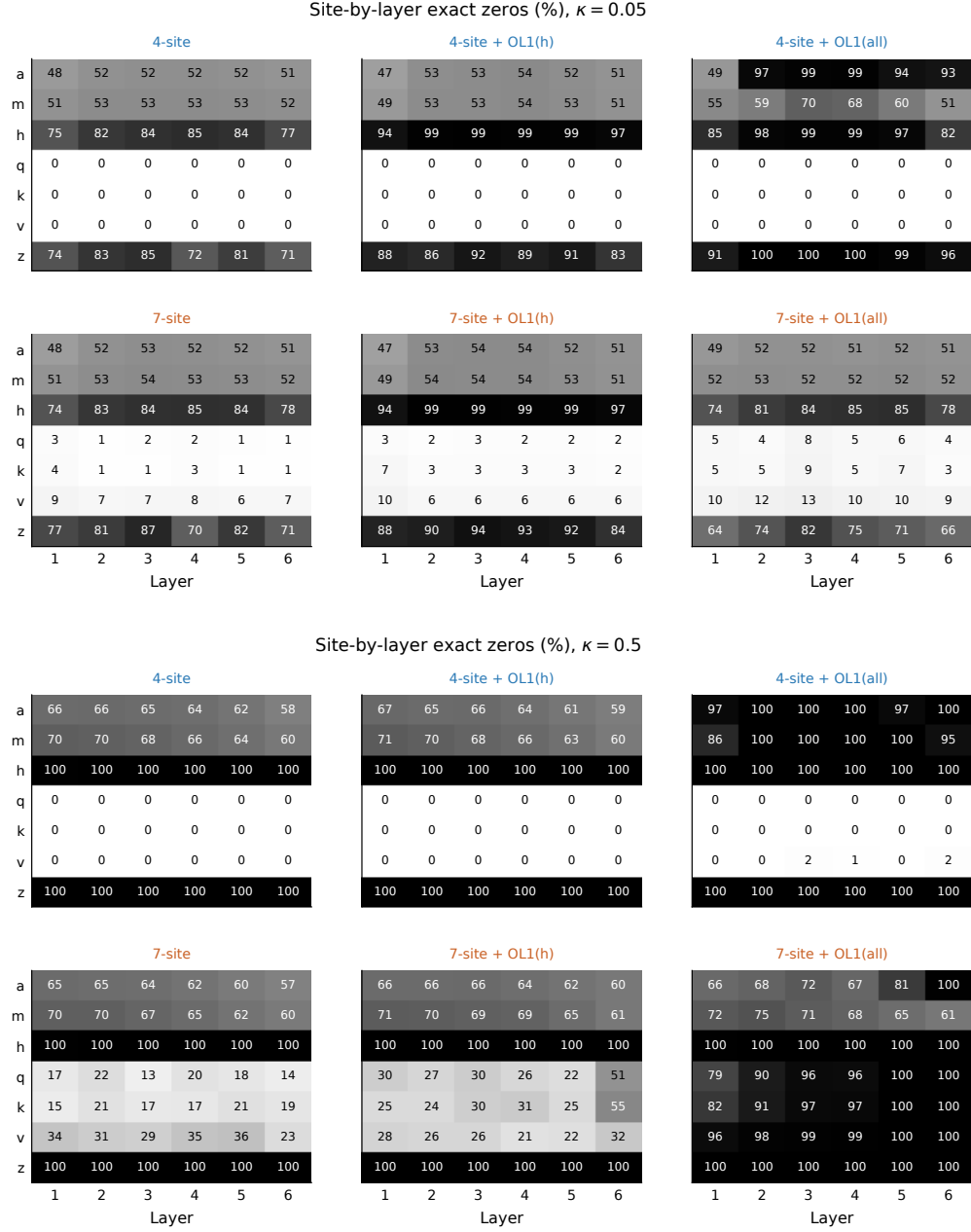


Figure 10: **Site-by-layer zero fractions under matched pressure scopes.** The six recipes at $\kappa = 0.05$ (top) and 0.5 (bottom). Each cell pools integer counts over all 338 validation blocks at one site and layer. Q/K are post-RoPE; values are rounded to whole percentages, so a displayed 100 does not imply an all-zero tensor. The complete five-threshold counters and source identities accompany the supplementary data.

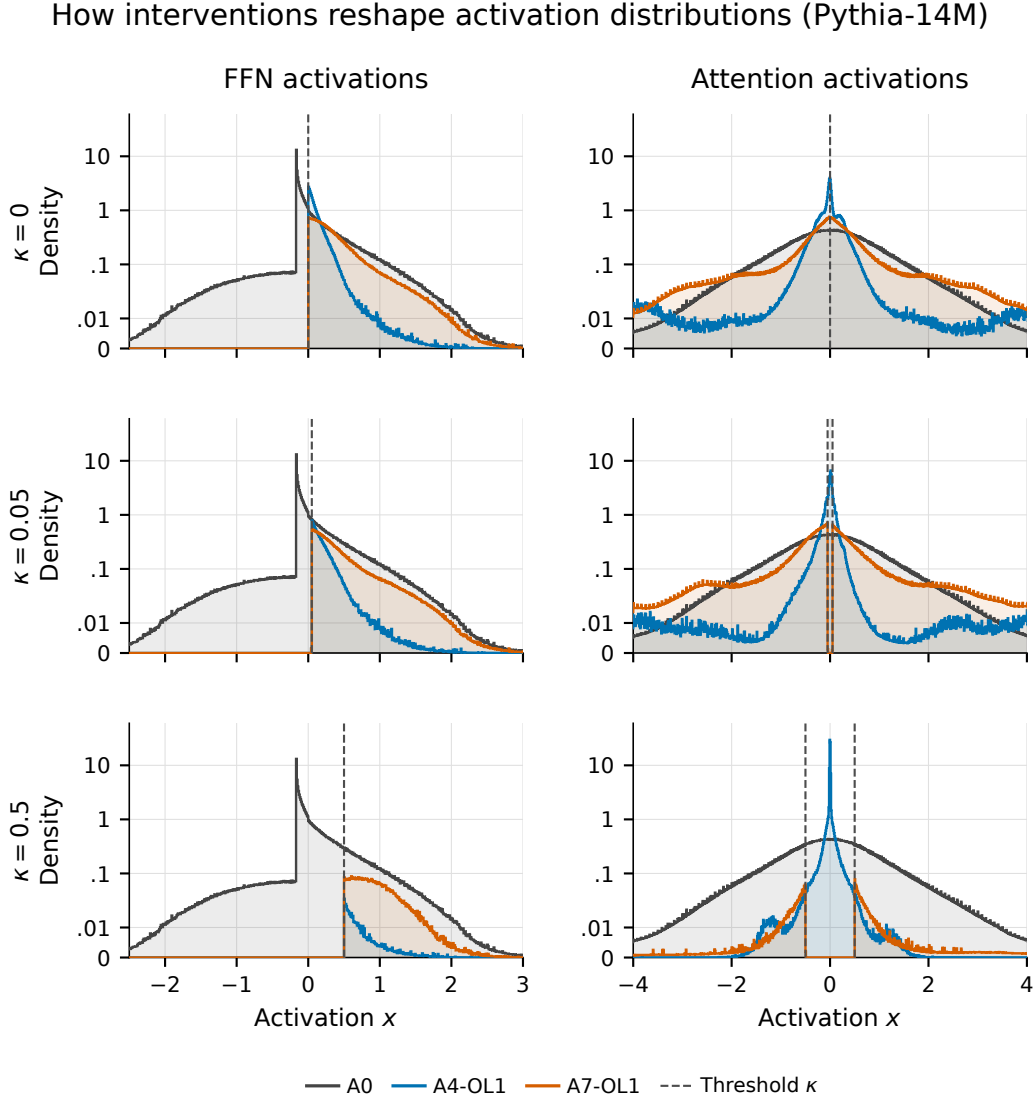


Figure 11: Interventions reshape FFN and attention distributions differently. Exact zeros are omitted from the curves and reported in Table 11. Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01; filled areas on this scale do not represent probability mass. Dashed lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc thresholding.

Table 11: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1(all)	0	FFN	67.22	0.0003	0.0000
A4-OL1(all)	0	Attention	< 0.01	5.1717	0.6162
A4-OL1(all)	0.05	FFN	86.71	0.0003	0.0000
A4-OL1(all)	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1(all)	0.5	FFN	99.31	0.0001	0.0000
A4-OL1(all)	0.5	Attention	0.22	0.0003	0.0000
A7-OL1(all)	0	FFN	64.52	0.0063	0.0000
A7-OL1(all)	0	Attention	< 0.01	5.4657	0.4648
A7-OL1(all)	0.05	FFN	75.29	0.0106	0.0000
A7-OL1(all)	0.05	Attention	7.26	6.6719	0.6932
A7-OL1(all)	0.5	FFN	93.64	0.0037	0.0000
A7-OL1(all)	0.5	Attention	95.60	0.2396	0.0009

D.2 OPERATION CONTRIBUTIONS TO MODEL-WIDE SPARSITY

An operation’s contribution to S_{model} is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 12 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1(all) to A7-OL1(all) adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation sparsity need not yield an equal change in model-wide sparsity. Figure 12 displays the operation-level view at the same threshold and all three sizes.

Table 12: Contributions to $100S_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1(all)	A7-OL1(all)	A4-OL1(all)	A7-OL1(all)	A4-OL1(all)	A7-OL1(all)
QKV	3.173	2.430	8.820	6.231	17.563	16.992
QK	0.000	8.324	0.000	5.052	0.000	5.206
PV	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

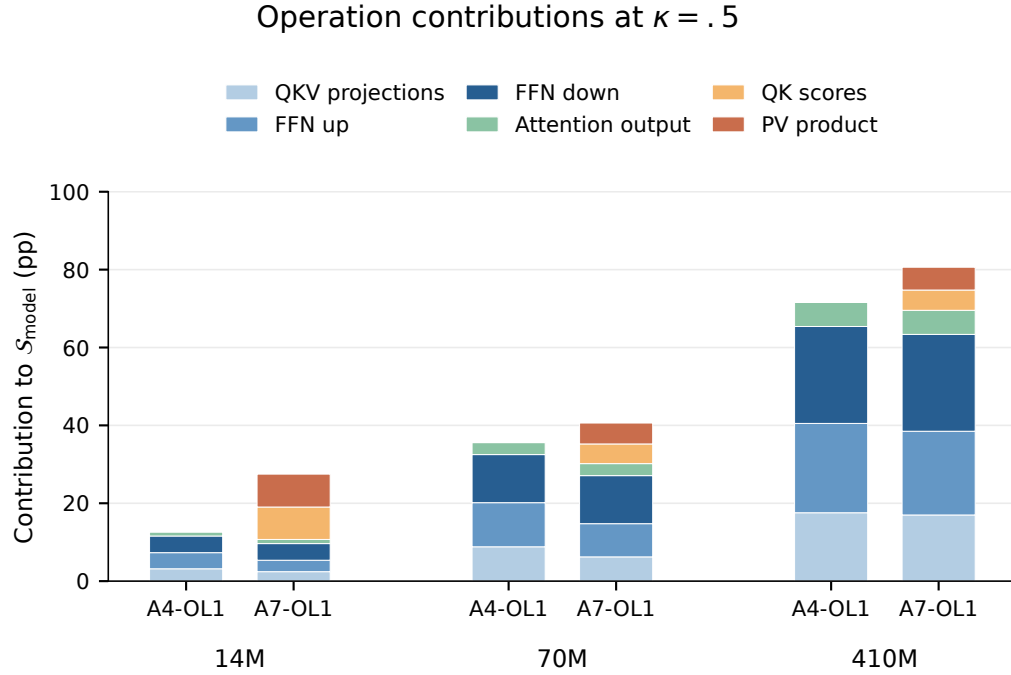


Figure 12: Operation accounting for A4-OL1(all) and A7-OL1(all) at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1(all), outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.3 COMPLETE HISTORICAL POST-HOC THRESHOLDING TRAJECTORIES

Historical plot legends retain “clipping” for post-hoc thresholding and “A4-OL1/A7-OL1” for the all-site pressure recipes. They do not include the newly integrated h-only conditions. The historical post-hoc release contains 540 evaluations: ten thresholding targets for each of the original 54 trained checkpoints. The ten newly included h-only endpoints have no corresponding post-hoc sweep. The release joins 350 evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 13–16 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

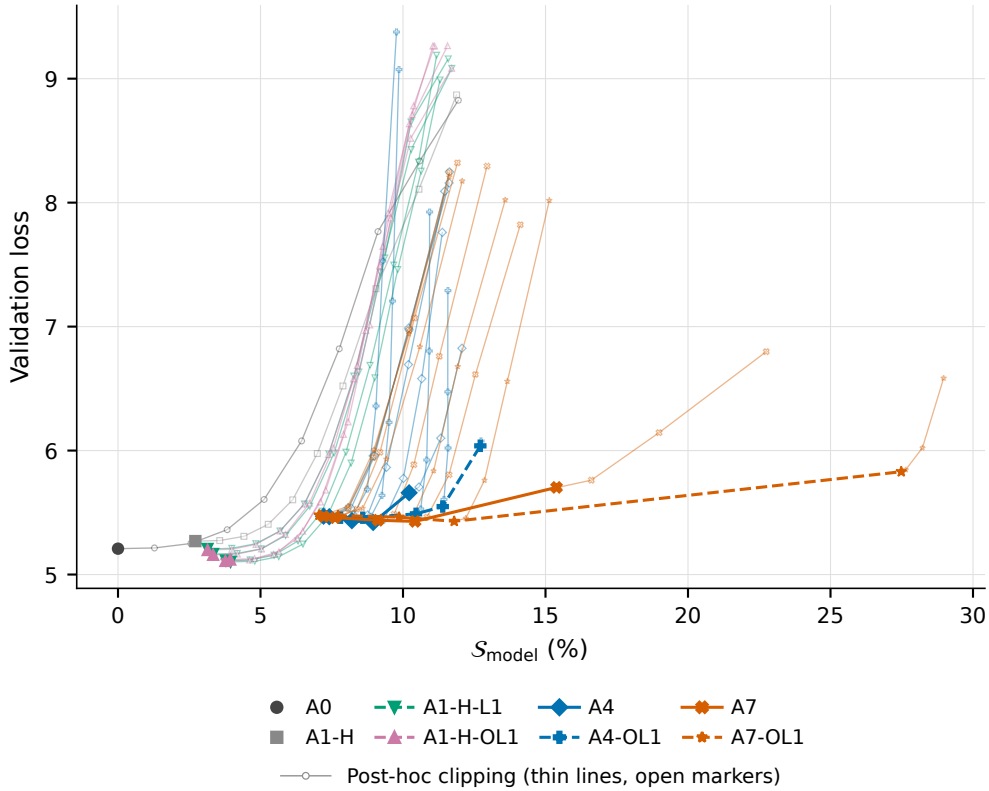


Figure 13: Complete 14M training and post-hoc thresholding results: 30 trained endpoints and 300 post-hoc thresholding evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

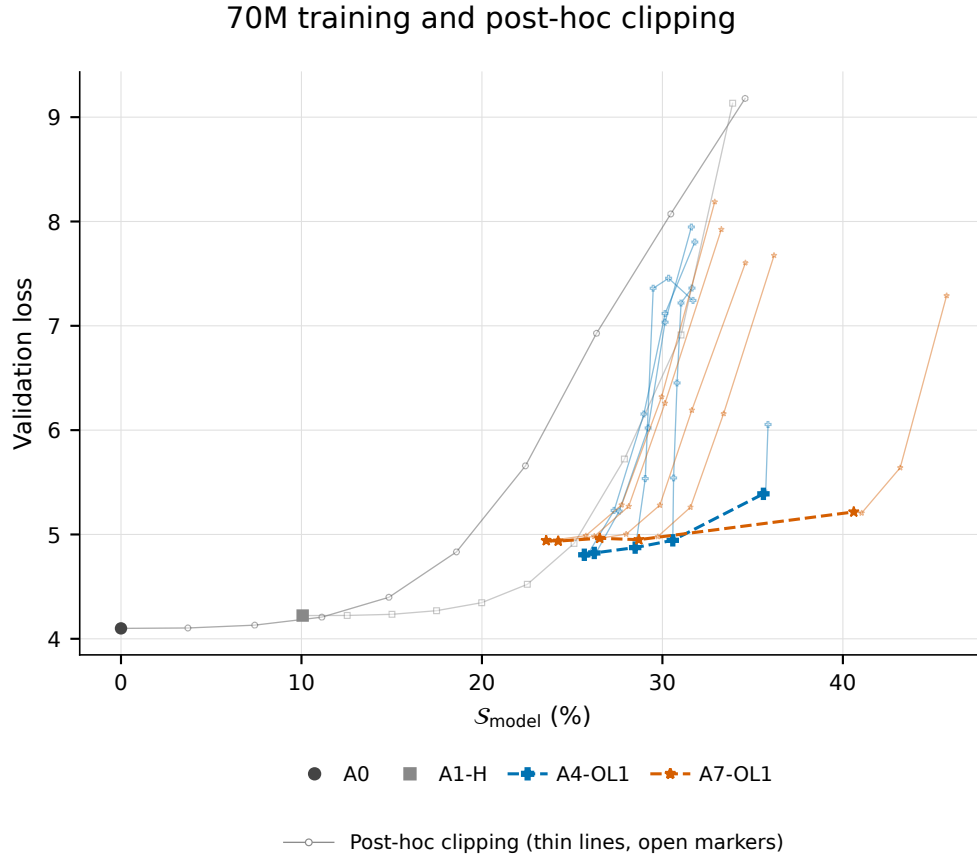


Figure 14: Complete 70M training and post-hoc thresholding results: 12 trained endpoints and 120 post-hoc thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing thresholding targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 13.

410M as a fixed-token stress test. Figure 15 retains all twelve trained 410M conditions and both complete control-clipping paths. At $\kappa = 0.5$, T_7/P_{all} reaches 80.62% model-wide sparsity, or 92.4% of the seven-site ceiling, at loss 5.121 versus the base model’s 4.547. This tests attainable sparsity under a larger architecture and the same token budget; it is not a quality-scaling experiment. The 410M seven-site ceiling is 87.25%, compared with 49.42% at 70M, so greater architectural reach also contributes to the larger observed model-wide sparsity.

The 410M base-model loss exceeds 70M’s 4.100. All sizes receive 712 optimizer steps and 1.493B input tokens, corresponding to 106.14, 21.20 and 3.68 tokens per parameter at 14M, 70M and 410M. The 410M schedule also has a lower peak learning rate (3×10^{-4} rather than 10^{-3}). These differences may contribute to the non-monotonic quality ordering, but their effects are not isolated. The training trajectories are retained in Figure 8. We therefore treat 410M as a limited stress test and use 14M/70M for detailed intervention and runtime analysis.

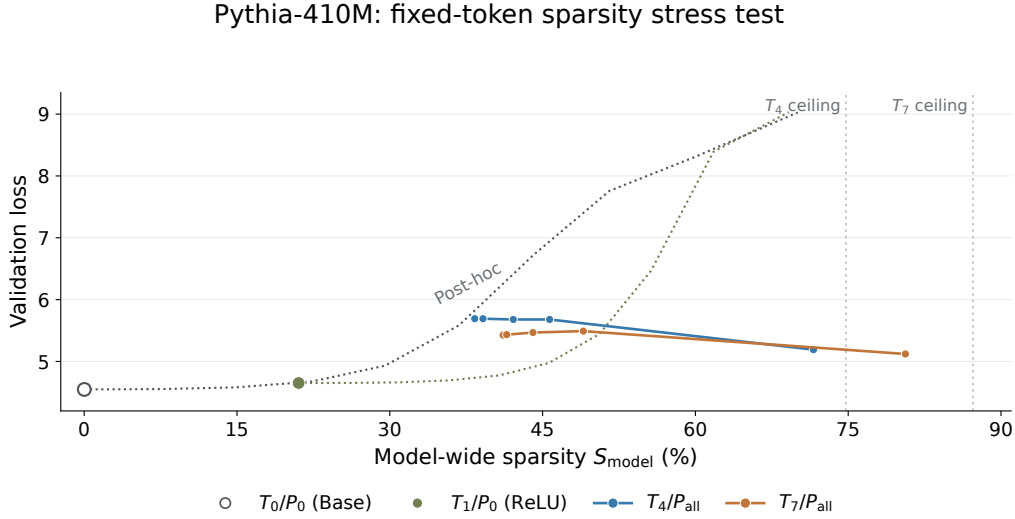


Figure 15: **Complete 410M quality–sparsity stress-test panel.** All twelve trained conditions: Base, ReLU, and T_4/P_{all} and T_7/P_{all} at $\kappa = 0, .01, .05, .1, .5$. Dotted paths retain all ten post-hoc targets for each fixed Base/ReLU control, including their full loss range. Vertical guides mark the four- and seven-site sparsity ceilings. Trained losses use ordinary final-checkpoint validation; clipping retains its original FP16 pass. Sparsity pools logical counts over all 338 complete validation blocks. Figure 16 additionally retains the historical clipping sweeps of every trained recipe.

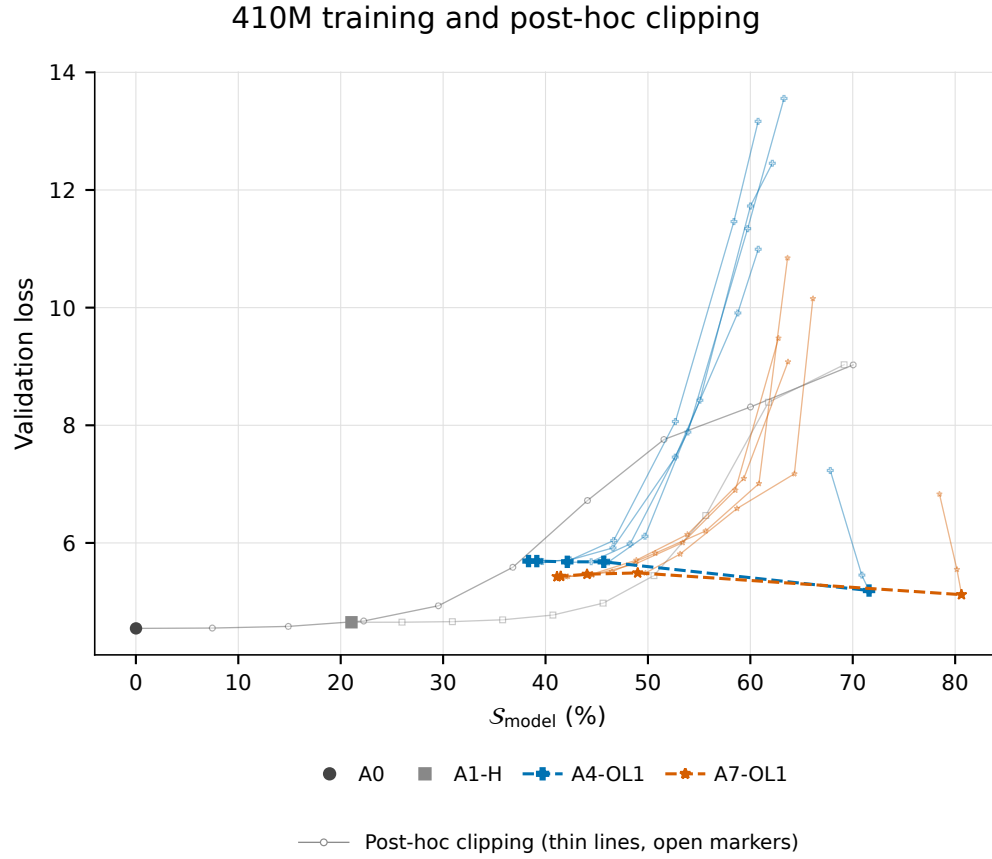


Figure 16: Complete 410M training and post-hoc thresholding results: 12 trained endpoints and 120 post-hoc thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing thresholding targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 13.

D.4 KERNEL DEVELOPMENT AND EVALUATION

Implementation. The specialized forward pass preserves each trained model’s weights, thresholds, biases and residual connections. It combines CUDA, CUTLASS and FlashAttention components to fuse operations and skip matrix instructions whose activation operands are zero. It introduces no additional pruning. One kernel computes the two input LayerNorms from their shared input while retaining separate affine parameters and thresholding. QKV layout conversion, RoPE and the selected q, k, v gates are fused; q, k gates follow RoPE. The FFN-down and attention-output projections are evaluated jointly with their biases and residual addition. The final vocabulary projection remains dense. Table 13 specifies the sparse execution rules.

Table 13: **Sparse execution rules.** Tile dimensions are tokens by input features. A matrix multiply-accumulate (MMA) instruction operates on $16 \times 8 \times 16$ elements (output rows, output columns, reduction dimension). Zero checks and any scalar replacement execute during every forward pass.

Operation	Execution rule	Work avoided
QKV, FFN-up (a, m)	Test each 16×16 activation fragment after both operands have been loaded.	Skip the MMA for an empty fragment; retain operand reads.
FFN-down, attention output (h, z)	Inspect eight rows; use scalar arithmetic for rows with at most two nonzeros when numerical guards pass. Process the remaining rows in 8×16 tiles padded to 16 rows.	Skip empty tiles before loading their weights. Scalar rows read only the corresponding weights.
Attention (QK, PV)	After loading the operands, use a warp-wide check to test whether either complete MMA fragment is zero.	Skip that MMA; retain input loading, masking, softmax and output processing.

The scalar path requires finite weights and biases, with nonzero magnitudes between 2^{-50} and 2^{50} ; selected activations obey the same magnitude bounds. Other rows use the matrix path. The two sizes retain these rules, with projection dimensions adapted from $(d, d_f) = (128, 512)$ to $(512, 2048)$. The 14M attention kernel uses 64×256 query/key tiles and two key/value partitions; 70M uses 128×128 tiles without partitioning to match its native attention reduction order. This change was needed for numerical agreement, rather than selected by a 70M performance search.

Scope and numerical checks. Tests use one RTX 5090, BF16 inference, batch size one, 2,048-token causal sequences, and all 50,304 output logits, with PyTorch 2.11.0, Transformers 5.12.1 and CUDA 12.8. The implementation assumes fixed weights, shapes and non-overlapping use of its buffers. It is not evaluated for training, cached decoding, concurrent serving, arbitrary masks or thresholds outside the tested grid. Each final checkpoint must pass in three fresh processes on all 338 validation sequences. Relative to the unmodified eager implementation, outputs must be finite, every logit must satisfy $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, the relative ℓ_2 error of each sequence’s logits must not exceed 0.02, and the absolute mean validation-loss difference must not exceed 0.001. These are numerical tolerances, not bitwise equivalence.

Timing and reference. Native PyTorch/SDPA and specialized inference use CUDA graphs with identical inputs. Latency is the geometric mean of host timings over 64 fixed validation sequences, seven randomized paired passes and three fresh processes. Compilation, static weight layout preparation, graph capture and equal input staging are excluded. Embeddings, all blocks, final normalization, vocabulary logits and input-dependent preprocessing are included. Post-hoc masks also remain inside timing; no activations are cached. Measurements use separate GPU sessions, so common normalization does not remove session effects.

For comparisons across recipes, Figure 17 uses *one native base-model latency per size*: $t_{\text{native}, T_0/P_0} / t_{\text{specialized}, r}$ for recipe r . The native references are 0.65544 ms at 14M and 1.66388 ms at 70M. This differs from Figure 4, which uses the specialized base-model latencies, 0.65157 and 3.31325 ms. The best trained-recipe gains are therefore $1.427\times$ and $1.028\times$ over the native base, versus $1.418\times$ and $2.047\times$ over the specialized base. The latter 70M ratio reflects a slow dense path in the ported implementation and does not establish a twofold gain over native inference.

What MMA bypass measures. Bypass is the number of omitted MMA instructions divided by issued plus omitted instructions, pooling integer counts across layers and validation sequences. Projection totals pool the four projection operations; attention totals pool QK and PV . We keep the two groups separate because equal instruction counts need not imply equal costs. At h, z , bypass includes replacement by scalar arithmetic and counts padded rows; it is not a fraction of all arithmetic eliminated. Attention counts include causal padding; even the base model bypasses 10.42% of PV instructions at 14M and 5.15% at 70M. In contrast, $\mathcal{S}_{\text{model}}$ excludes masked pairs and includes the dense vocabulary projection in its denominator.

The figure’s projection scalar sparsity pools zero-activation products over all projection products in the actual BF16 operands. Both this measure and bypass cover all 338 validation sequences; timing uses the 64-sequence subset. Thus these diagnostics complement, rather than redefine, the canonical model-wide sparsity used in the quality figures.

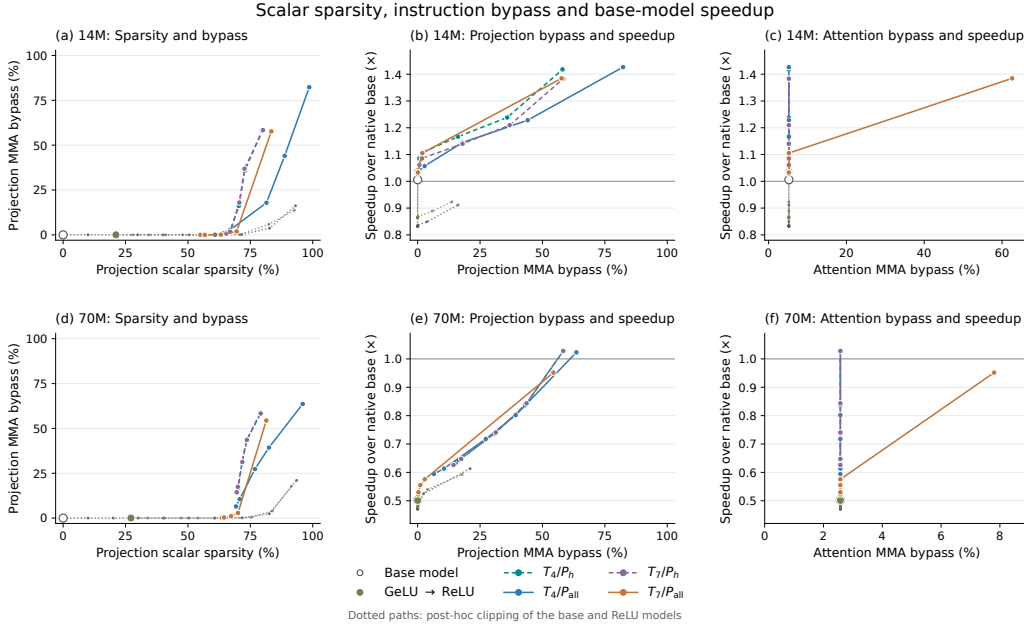


Figure 17: Scalar sparsity, instruction bypass and speedup over a fixed native base model. Rows show 14M and 70M, each with 22 trained settings and 20 post-hoc control settings. Left: projection scalar sparsity versus projection MMA bypass. Middle/right: projection/attention bypass versus full-model speedup. Every speedup within a row uses the same native T_0/P_0 reference; the horizontal line is $1\times$. The Base model marker uses specialized inference, so it need not lie on that line. Lines connect measured thresholds, not fitted trends; dotted paths are post-hoc clipping. Counts are pooled, include the instruction-padding and scalar-substitution conventions above, and do not measure saved memory traffic.

Interpretation and controlled tests. Projection bypass tracks speedup in both sizes, but high scalar sparsity alone need not produce empty instruction tiles. Post-hoc controls show this gap and incur separate masking costs. Attention bypass is nearly constant across the trained grid except for T_7/P_{all} at $\kappa = 0.5$; this single exception cannot establish a general attention-speedup relation. At 70M it has more attention bypass but less native-base speedup than T_7/P_h ($0.952\times$ versus $1.028\times$). Bypass therefore describes exploitable structure without explaining all latency differences.

Table 2 provides the direct 14M test: disable one skipping path while keeping weights, gates and fusion fixed. Its min-max spans cover all pairs of three independent off/on process means; effects are conditional and non-additive. These tests do not separately time memory reads, zero checks or softmax. No corresponding operation-level ablation was run at 70M, so its scatter is descriptive evidence, not causal attribution.

Alternatives tested. Porting the h, z strategy to a, m was correct but slower in the tested 14M case (Section 4.3). The padded fallback issued $1.90\times$ and $2.00\times$ as many matrix instructions, respectively. Attention shortcuts using cumulative value sums for zero queries, disjoint query/key supports, or cheaper conditional execution either failed numerical checks or did not establish a reliable gain. Disabling attention skipping improved all 35 historical 14M comparisons. The retained implementation is therefore not latency-optimal; it preserves one common execution policy across recipes. Earlier detection that avoids attention operand reads, fused post-hoc masking and sparsification of softmax probabilities remain untested here.